RELAZIONE TECNICA · PROGETTO FINALE LLM & AGENTIC AI

Casa & Regali — assistente clienti

Versione 2 (v2) · un agente, un passaggio di tool, un secondo passo LLM per la risposta

Abstract. Il progetto realizza un assistente clienti in italiano per un e-commerce fittizio di complementi d'arredo. L'LLM sceglie uno dei sette tool e scrive la risposta; un secondo passo LLM, senza tool, racconta l'esito. Le regole di dominio — finestra di reso, esclusioni, soglia di rimborso, privacy — sono codice deterministico. Il rimborso viene solo *proposto*: entra nel ledger unicamente dopo l'approvazione di un operatore. La v2 è la versione ottimizzata di un progetto esteso con supervisore, tre agenti, reflection e Chroma, e misura il risultato su 31 scenari con un modello locale da 1,7B.

31/31	100,0%	60	3,72 s
scenari superati	tool hit rate	test automatici	latenza media

1. Contesto e obiettivo

Un assistente clienti non è un problema di conversazione: è un problema di responsabilità. La risposta può essere scritta dall'LLM, ma un reso fuori finestra, un ordine di un altro cliente o un rimborso già eseguito non sono opinioni: sono fatti e regole. Se il modello sbaglia, l'errore non deve diventare una scrittura irreversibile.

Ho scelto un dominio chiuso: **10 documenti** di policy, **200 ordini** sintetici e **50 clienti**. Le richieste sono eterogenee — «quanto costa la spedizione?», «dov'è ORD-1001?», «posso restituire una tazza personalizzata?», «voglio il rimborso di ORD-1005» — ma finiscono tutte nello stesso punto: una risposta in italiano che deve usare i dati giusti, applicare le regole giuste e non superare i limiti del proprio ruolo.

L'obiettivo non è accumulare pattern. È costruire un turno che risponda con fonti verificabili, usi solo gli ordini del cliente della sessione, applichi la policy in modo deterministico e chieda l'approvazione di un umano prima di un'azione sensibile. Il criterio di successo è misurabile: tool corretto, risposta corretta, nessun accesso a ordini altrui, nessun rimborso senza approvazione, nessun effetto collaterale indesiderato.

- **Rispondere** in italiano, citando `[S1]`, `[S2]` quando la risposta viene dal corpus.
- **Usare** solo gli ordini del profilo della sessione, mai un cliente scelto dal modello.
- **Applicare** finestra di reso, esclusioni e soglie con codice deterministico.
- **Chiedere** l'approvazione umana prima di scrivere un rimborso.

2. La tesi: due decisioni all'LLM, i vincoli al codice

La domanda di progetto è stata: **quale parte del sistema ha davvero bisogno dell'LLM?** La risposta è duplice. L'LLM serve per scegliere il tool e per scrivere una risposta in linguaggio naturale. Tutto il resto — idoneità al reso, soglie di rimborso, privacy, approvazione, scrittura del ledger — è codice deterministico e verificabile.

Ho scelto di separare la *scelta* dalla *narrazione*: il nodo `agent` decide, il nodo `tools` esegue, il nodo `respond` scrive senza poter avviare altre azioni. In questo modo l'LLM prende due decisioni osservabili, ma non può concatenare azioni né aggirare un vincolo di dominio.

> **La scelta di progetto**
>
> Non c'è una modalità deterministica che sostituisca il modello quando sbaglia: gli errori restano visibili nella valutazione. È una scelta più onesta e più leggibile di un fallback che maschera il comportamento reale.

2.1 Cosa fa il modello, cosa fa il codice

Compito	Responsabile
Scegliere il tool e i suoi argomenti	LLM
Scrivere la risposta al cliente	LLM
Applicare finestra resi, esclusioni e soglie	`policy.py`
Garantire il cliente della sessione	`ToolRuntime` + `tools.py`
Sospendere il turno per l'approvazione	`interrupt()`
Scrivere il rimborso solo se approvato	`audit.py` (ledger)
Registrare turni, accessi negati e ticket	`audit.py`

Policy-as-code. La fonte di verità è `policy.py`: **30 giorni** di reso, categorie escluse (personalizzati, igiene, digitali, fine serie), soglia di escalation di **500 EUR**, prima risposta di un ticket entro **24 ore**, calcolo del rimborso e controllo duplicati. I documenti *spiegano* la policy, i tool la *applicano*; un test confronta i numeri dei documenti con le costanti del codice.

Privacy. Il `customer_id` arriva dallo stato del grafo tramite `ToolRuntime`: il modello non sceglie di quale cliente parlare. Un ordine altrui restituisce lo stesso messaggio di un ordine inesistente, e l'accesso negato finisce nell'audit. I dati del profilo non vengono passati al modello: i tool restituiscono solo i campi dell'ordine necessari.

3. Architettura e un turno

Il grafo ha quattro nodi: **agent** sceglie un tool o risponde; **tools** esegue il tool con il `ToolRuntime` della sessione; **respond** è un secondo passo LLM senza tool che scrive la risposta; **finalize** salva risposta, fonti e tool usati e scrive il turno nell'audit. Se non serve un tool, il percorso è `agent` → `finalize`.

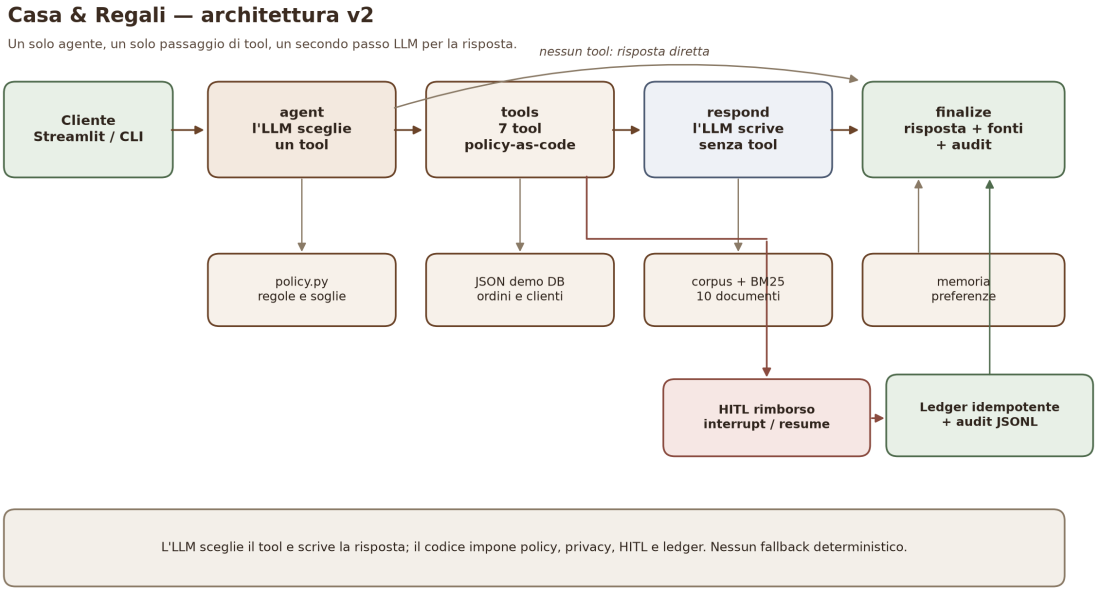


Figura 1 — Flusso agent → tools → respond → finalize. La freccia superiore è il percorso senza tool; il ramo rosso è il rimborso con HITL e ledger.

Ho scelto un solo passaggio di tool per turno. Il ciclo ReAct completo (`agent` → `tools` → `agent` → …) con modelli da 1-3B tende a ripetere lo stesso tool o a concatenarne altri dopo un esito già definitivo. Separare scelta e narrazione riduce latenza e catene incontrollate.

Un turno completo. «Voglio il rimborso di ORD-1001»: `agent` chiama `proponi_rimborso("ORD-1001")`; il tool verifica il cliente, applica la policy e calcola **89,70 EUR**. `interrupt()` sospende il turno con una proposta a livello *operatore*; la UI riprende con `Command(resume=...)` e il ledger rivalida cliente, importo e duplicati prima di scrivere. `respond` racconta l'esito, `finalize` registra il turno; se l'operatore rifiuta, non viene scritto nulla.

4. Mappa del codice

La tabella collega ogni responsabilità al file che la implementa: è il livello di lettura del codice, complementare al flusso descritto nelle sezioni 2 e 3.

File	Responsabilità	Dettagli da conoscere
agent.py	Grafo e turno	`SupportState` (`messages`, `customer_id`, `final_answer`, `sources`); quattro nodi; `respond` senza tool; `finalize` legge solo il turno corrente e ha un fallback; `start_turn/resume_turn`.
tools.py	Sette tool	`ToolRuntime` inietta il `customer_id`; stesso messaggio per ordine inesistente o altrui; `cerca_policy` unisce query e richiesta; `proponi_rimborso` sospende e scrive solo dopo l'approvazione.
policy.py	Regole deterministiche	30 giorni, 24 mesi, 5-10 giorni, 50/4,90 EUR, soglia 500 EUR; `find_order/list_orders` filtrati; albero di idoneità; `calculate_refund` con spedizione rimborsabile.
audit.py	Prove e ledger	Eventi JSONL (turno, privacy, ticket, rimborsi); `execute_refund` ricontrolla approvazione, ordine, cliente, importo e duplicati; lock e file temporanei.
prompts.py + turns.py	Qualità della risposta	Prompt per tool; contratto `<risposta>`; `extract_answer`; helper per messaggi, turno, tool e fonti.
corpus.py	RAG BM25	10 documenti; split per sezioni; chunk 650/50; BM25 e sinonimi; top 3 con score > `0`; fonti `[S1]`.
memory.py + models.py + config.py	Runtime	Preferenze esplicite, PII rimosse, duplicati ignorati; reasoning finale spento per LFM3 locale; `Settings`, `demo_today` e percorsi.
app.py + cli.py + eval/ + tests/	Interfacce e verifica	`session_state` e pending; conferma > 500 EUR; ciclo interrupt nella CLI; 31 scenari su file temporanei; metriche; `ScriptedModel`.

I tool portano nel turno dati e controlli deterministici: il prompt descrive le regole, il codice le applica e il ledger le rivalida prima di ogni scrittura.

5. Scelte e alternative scartate

Scelta	Alternativa scartata	Perché	Conseguenza
Un solo agente	Supervisore con agenti specialistici	Sette tool e un dominio: la divisione in specialisti moltiplica le chiamate senza aggiungere capacità	31/31, 0,94 tool per turno
Un passaggio di tool, poi `respond`	Ciclo ReAct completo	Con modelli piccoli il ciclo tende a ripetere il tool o a concatenarne altri dopo un esito definitivo	Nessun tool dopo il rimborso
BM25	Chroma con embedding	Dieci documenti brevi: BM25 è deterministico, non richiede indice né modello di embedding	Source hit 100%
Policy-as-code	Regole nel system prompt	Il prompt può essere ignorato o attaccato con prompt injection; il codice no	Prompt injection superata
HITL nel tool	Nodo di approvazione separato	Proposta e approvazione restano nello stesso punto; un nodo separato lascerebbe altri percorsi di scrittura	Rimborso pulito 100%
Eval custom e audit JSONL	LangSmith	La suite è piccola e il progetto resta riproducibile senza account esterni; il JSONL copre turni, privacy e rimborsi	31 scenari e failure gallery

Prompting. Il system prompt usa ruolo e vincoli, few-shot e output strutturato: lo schema degli argomenti è generato da LangChain, quindi il modello non inventa campi. Il prompt di scelta resta breve; quello finale riceve solo richiesta ed esiti, cambia per tool e chiede un blocco `<risposta>`. Il codice estrae solo quel blocco, così il ragionamento non arriva al cliente.

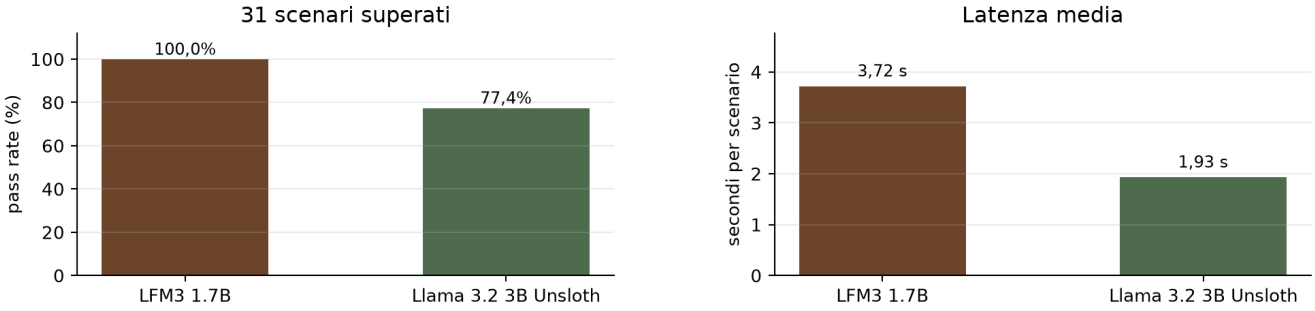
Perché non multi-agente, reflection o LangSmith. Un dominio, sette tool e un turno singolo non li richiedono: aggiungerebbero passaggi, latenza o un account esterno senza migliorare la scelta.

Un fallimento concreto. Nella run Unsloth, `info_personalizzato` chiama `verifica_reso` invece di `cerca_policy`; `memoria_implicita` salva una preferenza non richiesta. La failure gallery conserva gli errori.

6. Valutazione

La suite contiene **31 scenari**: 9 informazioni, 3 ordini, 6 resi, 6 rimborsi, 3 sicurezza (privacy, ordine inesistente, prompt injection), 2 escalation e 2 memoria. Ogni scenario verifica, dove previsto, tool attesi e vietati, contenuto, fonti, interrupt HITL, scritture e assenza di effetti collaterali. I rimborsi usano dati temporanei, senza toccare la demo.

Profilo	Superati	Pass rate	Tool hit	Fonti	HITL	Sicurezza	Latenza
LFM3-1.7B Q4_K_M	31/31	100,0%	100,0%	100,0%	100,0%	100,0%	3,72 s
Llama 3.2 3B Unsloth	24/31	77,4%	96,6%	88,9%	100,0%	66,7%	1,93 s



LFM3 è più lento ma più affidabile: **31/31** con **0,94 tool per turno** e 3,72 s medi. Llama (Unsloth) è più veloce (1,93 s) ma si ferma a 24/31: sbaglia tool, fonti e sicurezza (66,7% contro 100,0%); i 7 fallimenti si concentrano su resi, memoria e sicurezza. Il modello più piccolo non è un dettaglio: è il vincolo che rende necessari i confini di codice.

Reasoning. Con LFM3 il reasoning attivo (variabile vuota) ha dato la migliore scelta dei tool. La risposta finale lo disattiva sempre: con la tabella dei 10 ordini il secondo passo consumava **7.058 token** e restituiva contenuto vuoto. Con `LM_STUDIO_REASONING_EFFORT=none` su tutto il grafo una run precedente si è fermata a **27/31**. **Gemini.** Il backend cloud è presente, ma non c'è una run completa per esaurimento della quota: non riporto una metrica che non ho misurato.

Metodo. La valutazione misura il sistema così com'è: se il modello sceglie il tool sbagliato o inventa un numero, lo scenario fallisce. Non c'è una correzione a posteriori che nasconda l'errore; la failure gallery conserva risposta, tool e controlli falliti. La suite automatica usa un modello scriptato (`ScriptedModel`) e dati isolati: **60 test** senza LM Studio né rete, con fixture temporanee e data fissa. Coprono flussi del grafo, tool, policy/corpus, ledger, memoria, modelli, helper e coerenza degli scenari; non misurano la qualità del modello, che resta compito della valutazione.

7. Limiti e conclusione

- **Modelli piccoli.** LFM3 1,7B e Llama 3.2 3B (Unsloth) possono sbagliare la scelta del tool o riformulare i numeri; la valutazione misura questi errori e non li corregge con un fallback.
- **Un tool per turno.** Una richiesta che ne richiede due viene gestita al turno successivo: è una scelta deliberata, non un limite tecnico del grafo.
- **BM25.** Adatto a dieci documenti brevi; su un corpus più grande servirebbe ricerca semantica o ibrida.
- **Memoria di sessione.** `InMemorySaver` perde lo stato al riavvio; le preferenze a lungo termine restano su file.
- **Demo a processo singolo.** Ledger e memoria usano lock di processo e file JSON, adatti alla demo ma non a un deployment multi-istanza.

La v2 mantiene i vincoli di dominio e toglie i pattern che, con un modello da 1-3B, non ripagavano la complessità. Il grafo è piccolo abbastanza da essere letto in un minuto, ma il comportamento non è simulato: l'LLM sceglie davvero il tool, scrive davvero la risposta e sbaglia quando il modello sbaglia. La differenza è che gli errori sono visibili nella valutazione, non nascosti da un fallback.

Il 31/31 di LFM3 non è una garanzia di produzione: è la misura di un prototipo su 31 scenari controllati, con un modello locale e dati sintetici. Llama 3.2 3B (Unsloth) mostra che il margine di errore esiste e dipende dal modello. Il secondo passo LLM separato riduce però il perimetro degli errori: la risposta può essere imprecisa, ma non può avviare un rimborso o cambiare cliente.

Appendice A — Dettaglio delle funzioni Python

Il corpo della relazione resta di quattro pagine; questa appendice è il livello di lettura funzione per funzione. Le schede seguono il percorso del turno e riportano input, output, comportamento, casi limite e test. La versione completa, con tutte le funzioni di supporto, è nel README (§15).

A.1 Grafo e turno — agent.py

Funzione	Input → output	Comportamento e casi limite	Test
`build_graph(...)`	Settings e dipendenze opzionali → grafo compilato	Crea tool e modelli, collega i tool, costruisce i quattro nodi e gli archi, usa `InMemorySaver` se manca un checkpointer. Per LFM3 locale crea un secondo modello con reasoning spento per `respond`.	Tutti i flussi; models
`agent_node` + `route_agent`	stato → `AIMessage` / `"tools"` o `"finalize"`	Antepone il system prompt e chiama il modello con i tool. Se l'ultimo messaggio ha `tool_calls` va a `tools`, altrimenti chiude. Non esiste il ritorno da `tools` ad `agent`.	graph
`respond_node`	stato → `AIMessage`	Legge l'ultima richiesta e i risultati dei tool, sceglie il prompt per tool e chiama il modello senza tool. `extract_answer` scarta il ragionamento fuori da `<risposta>`; un contenuto vuoto attiva il fallback.	graph, turns
`finalize`	stato → `final_answer`, `sources`	Cerca l'ultima AI del turno corrente, non del precedente. Se è vuota usa il messaggio neutro, raccoglie fonti e tool e scrive l'evento `turno` con risposta troncata a 500 caratteri.	graph
`start_turn` / `resume_turn`	grafo, thread, input → risultato	Il primo azzera risposta e fonti e invoca il grafo con `recursion_limit=20`; il secondo riprende l'interrupt con `Command(resume=...)`.	graph, HITL
`pending_interrupt` / `state_values`	risultato/grafo → dict	Il primo legge `__interrupt__` e normalizza oggetto o dizionario; il secondo restituisce i valori dello stato del thread.	graph

A.2 Tool e dominio — tools.py, policy.py

Funzione	Input → output	Comportamento e casi limite	Test
`make_tools` / `_customer`	ledger, memoria, settings → sette tool	I tool sono chiusi sulle dipendenze della sessione; `_customer` legge il cliente dallo stato del grafo, mai dagli argomenti del modello.	tools, eval
`cerca_policy`	query → JSON con contesto e fonti	Unisce la query all'ultima richiesta umana, chiama `retrieve` e restituisce fonti `[S1]`–`[S3]`.	tools
`stato_ordine` / `elenca_ordini`	ordine o runtime → testo	Filtrano per cliente. Un ordine altrui e uno inesistente producono lo stesso messaggio; l'elenco non mostra mai identificativi o contatti del profilo.	tools
`verifica_reso`	ordine → testo	Applica la policy con la data demo, riporta giorni, categorie escluse e difettosità. Un ordine non trovato passa prima dal controllo privacy.	tools, policy
`proponi_rimborso`	ordine → testo o interrupt	Verifica idoneità e cliente, calcola l'importo, costruisce la proposta e chiama `interrupt`. Un rifiuto non scrive; un'approvazione passa al ledger. Un ordine non idoneo non apre nulla.	graph, HITL
`apri_ticket` / `ricorda_preferenza`	testo → testo	Il primo calcola la priorità da parole di escalation e crea il ticket; il secondo salva solo preferenze esplicite, con PII mascherate e duplicati ignorati.	tools, memory
`find_order` / `list_orders`	ID/cliente → dict o lista	Normalizzano l'ID e filtrano sempre per cliente. `find_order` non distingue un ordine altrui da uno inesistente; la lista è ordinata dal più recente.	policy
`check_return_eligibility`	ordine, cliente, data → dict	Controlla cliente, rimborso, stato, consegna, data, finestra di 30 giorni ed esclusioni. Restituisce motivo, giorni, articoli/categorie esclusi e riferimenti alla policy.	policy
`calculate_refund`	ordine → dict	Somma articoli, sottrae lo sconto e aggiunge la spedizione solo se il reso è a carico nostro. Arrotonda a due decimali; l'escalation scatta sopra 500 EUR.	policy
`execute_refund`	proposta, approvazione → dict	Unico punto di scrittura. Ricontrolla approvazione, ordine, cliente, importo e duplicati; scrive refunds e orders con file temporanei e `os.replace`.	audit

A.3 Dati, risposta e verifica — gli altri moduli

Funzione	Input → output	Comportamento e casi limite	Test
`AuditLedger.log` / `read_audit`	evento/limite → record/lista	Scrive JSONL sotto lock, scarta i campi None e ignora le righe malformate in lettura. Gli eventi coprono turni, privacy, ticket e rimborsi.	audit
`create_ticket` / `_next_refund_id`	dati → ticket/ID	Il ticket usa `TKT-2026-NNNN` e salta i buchi; il rimborso usa il suffisso numerico più alto e restituisce `RMB-2026-NNNN`.	audit
`load_corpus` / `split_documents`	cartella/documenti → chunk	Legge i dieci Markdown, separa per titolo, sezione e sottosezione, poi applica chunk 650/50 e assegna un `chunk_id` univoco.	corpus
`retrieve` / `format_context`	query → documenti/testo	BM25 ricostruito a ogni query, top-k con score positivo, sinonimi italiani e fonti `[S1]–[S3]` numerate.	corpus
`_sanitize` / `PreferenceMemory.add`	testo → record o esito	Maschera email, telefoni e carte, tronca a 200 caratteri, ignora duplicati e salva con sostituzione atomica. Una preferenza implicita non viene salvata dal prompt.	memory, tools
`get_chat_model` / `final_reasoning_effort`	settings → modello/str	Gemini richiede la chiave; locale usa temperatura 0, timeout 120 e un retry. Il reasoning finale è `"none"` solo per LFM3 locale.	models
`extract_answer` / `sources_from_messages`	testo/messaggi → risposta/fonti	Il primo isola `<risposta>` e lascia il testo se i tag mancano; il secondo interpreta il JSON di `cerca_policy`, deduplica e limita a 6 fonti.	turns
`eval.run_scenario` / `summarize`	scenario/risultati → dict	Ogni scenario usa file temporanei; verifica tool, risposta, fonti, HITL, scritture e memoria. Il riepilogo calcola le metriche solo sugli scenari che prevedono quel controllo.	eval

Le etichette della colonna **Test** rimandano ai file `tests/test_*.py`: *graph*, *tools*, *policy*, *audit*, *corpus*, *memory*, *models*, *turns* ed *eval*. La mappatura completa, test per test, è nel README §15.

Le funzioni che possono cambiare lo stato sono cinque: `proponi_rimborso`, `execute_refund`, `create_ticket`, `PreferenceMemory.add` e `finalize` per l'audit. Per ognuna la domanda orale è la stessa: quali controlli precedono la scrittura e quale test dimostra che un errore non la esegue.

A.4 Tre funzioni raccontate

Le tabelle precedenti danno la mappa; queste tre schede mostrano il livello di dettaglio utile all'orale. Sono le funzioni su cui si concentrano le domande su privacy, HITL e policy.

proponi_rimborso(order_id, runtime)

Input → output. Numero ordine e runtime della sessione → testo con l'esito, oppure sospensione del turno con una proposta HITL.

- Legge il cliente dallo stato con `_customer` e cerca l'ordine con `find_order`: un ordine altrui o inesistente produce lo stesso messaggio e un evento privacy.
- Applica `check_return_eligibility` con la data demo. Se l'ordine non è idoneo, restituisce il motivo e non apre alcun interrupt.
- Calcola l'importo con `calculate_refund` e costruisce la proposta con ordine, cliente, importo, motivo, articoli e livello.
- Chiama `interrupt({"type": "refund_approval", "proposal": ...})`: da qui il turno è sospeso e nessuna scrittura è avvenuta.
- Dopo la ripresa, un rifiuto registra `rimborso_rifiutato` e restituisce un testo; un'approvazione chiama `execute_refund`.
- Traduce l'esito del ledger in testo: eseguito, duplicato o errore. Non scrive mai direttamente.

Casi limite e test. Un ordine non idoneo non produce interrupt; un'approvazione mancante equivale a rifiuto; un secondo tentativo viene bloccato dal ledger. I test sono `test_rimborso_sospende_e_non_scrive`, `test_approvazione_scrive_una_sola_volta` e `test_rifiuto_non_scrive`.

execute_refund(proposal, approval, *, thread_id=None)

Input → output. Proposta, decisione umana e thread → dizionario con `eseguito`, `duplicato`, `motivo` e, se presente, `rimborso`.

- Controlla `approval["approved"]`: senza approvazione registra il rifiuto e restituisce `eseguito=False`.
- Normalizza ordine, cliente e importo e acquisisce il lock; rilegge refunds e orders dal disco.
- Blocca il duplicato sia sulla chiave ordine sia su un record storico con lo stesso ordine.
- Verifica che l'ordine esista, che il cliente coincida e che l'importo corrisponda a `calculate_refund` entro 0,01 EUR.
- Genera `RMB-2026-NNNN`, aggiorna refunds e orders, sostituisce i file con `os.replace` e registra `rimborso_eseguito`.

Limite dichiarato. La coppia di file non è una transazione unica: un crash tra le due sostituzioni può lasciare i due file disallineati. Il lock è di processo, quindi la demo non è pensata per più istanze. I test `test_senza_approvazione_non_scrive`, `test_importo_non_coerente_bloccato` e `test_idempotenza` coprono i confini principali.

check_return_eligibility(order, customer_id, *, today, refunds_path=None)

Input → output. Ordine, cliente e data di riferimento → dizionario con idoneità, motivo, giorni dalla consegna, articoli e categorie esclusi e riferimenti alla policy.

- Rifiuta un ordine di un altro cliente: è una difesa ridondante rispetto al filtro del tool.
- Rifiuta un ordine già rimborsato, annullato o non ancora consegnato: la finestra decorre dalla consegna.
- Calcola i giorni con `days_since_delivery`; una data mancante o futura produce un motivo dedicato.
- Se i giorni superano 30, restituisce il numero esatto e la finestra. Se sono entro 30, controlla gli articoli: basta un articolo personalizzato, di igiene, digitale o in fine serie per bloccare.
- Restituisce sempre un esito spiegabile; non scrive nulla e non decide l'importo.

Test. `test_idoneita_finestra_e_categorie` copre i sei ordini curati; `test_motivi_idoneita` controlla i motivi; `test_verifica_reso_usa_la_data_delle_settings` dimostra che la data non è letta dall'orologio di sistema.